

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: CRUD de Estudiante

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Gestión de CRUD de empleados con los datos ID, Nombre , Rol, Correo
Operaciones CRUD	Agregar, Actualizar, Eliminar y Mostrar Todo
Referencia base	01_Guia_PlantUML_CU_Analisis_CRUD_TekMess_V1.0

1. Propósito de la guía

Esta guía complementa el modelado inicial de casos de uso del sistema **TekMess**. Su finalidad es demostrar cómo el requisito funcional **RF-01: Gestionar empleado** (que permite administrar la información del personal operativo como guardias, supervisores y centralistas) puede analizarse de forma progresiva mediante tres modelos técnicos conectados: **casos de uso de análisis**, **clases de análisis** y **diagramas de secuencia de análisis**.

El énfasis de este documento no es la programación de la interfaz, sino la comprensión de las responsabilidades de cada elemento del modelo (Frontera, Control y Entidad) y asegurar la **trazabilidad** completa desde la necesidad expresada por el **Jefe de Logística** en la ERS hasta la interacción detallada entre los objetos del sistema.

A través de esta metodología, se busca garantizar que cada acción realizada por el actor principal en el módulo de empleados sea validada técnica y dinámicamente antes de su implementación en el entorno de desarrollo

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el Jefe de Logística para administrar al personal operativo?	Diagrama de casos de uso y matriz de trazabilidad RF-CU	Permite identificar las responsabilidades del sistema y definir las clases frontera, control y entidad necesarias para cumplir con el RF-01
Clases de análisis	¿Qué objetos participan internamente para registrar o editar a un empleado?	Diagrama de clases con estereotipos: boundary (interfaz), control (lógica), entity (datos del empleado) y repository (persistencia en PostgreSQL)	Define los objetos y métodos específicos que aparecerán e interactuarán en los diagramas de secuencia
Secuencias de análisis	¿Cómo colaboran el actor y los objetos para ejecutar el registro de un nuevo guardia o supervisor?	Diagramas de secuencia por operación (Crear, Editar, Eliminar y Generar credenciales)	Valida que el flujo de mensajes sea coherente con los requisitos de seguridad (como la validación de datos y roles) antes de la implementación en Dart/Flutter

3. Caso base actualizado: Gestión de Empleados (TekMess)

El módulo se basa en una interfaz denominada "Gestión de Personal Operativo", diseñada para que el Jefe de Logística administre a los guardias, supervisores y centralistas

Dato	Tipo sugerido	Descripción	Validación mínima
Cédula	Texto corto o entero	Identificador legal único del empleado en Ecuador	Obligatorio, 10 dígitos, no debe existir previamente en la BD.
Nombre	Texto	Nombres y apellidos completos del trabajador	Obligatorio, no debe contener caracteres especiales o números.
Rol (RBAC)	Enum / Lista	Perfil asignado: Guardia, Supervisor, Centralista o Jefe	Obligatorio, debe pertenecer a uno de los 4 perfiles definidos.
Correo	Texto	Dirección electrónica para el envío del código OTP y credenciales	Obligatorio, formato de email válido y único.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Registrar	Crear empleado	Captura los datos, valida la cédula y persiste al nuevo usuario en la base de datos PostgreSQL
Actualizar	Editar empleado	Permite modificar nombres, correo o rol de un empleado existente seleccionado por su cédula
Dar de baja	Eliminar empleado	Cambia el estado del empleado a "Inactivo" para revocar su acceso al sistema de mensajería
Listar Personal	Mostrar empleados	Recupera de la base de datos la lista de todo el personal operativo registrado para auditoría
Generar Acceso	Generar credenciales	Activa el envío automático de la contraseña temporal y habilita el canal de comunicación

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01.01	El sistema debe permitir registrar un nuevo empleado (Guardia, Supervisor, Centralista).	Jefe de Logística	Crear empleado
RF-01.05	El sistema debe permitir actualizar la información del personal registrado.	Jefe de Logística	Editar empleado
RF-01.06	El sistema debe permitir la eliminación o baja de un empleado del sistema.	Jefe de Logística	Eliminar empleado
RF-01.03	El sistema debe generar automáticamente las credenciales de acceso tras el registro.	Sistema	Generar credenciales
NF-02	El sistema debe validar que los datos cumplan con los estándares de seguridad (Cédula, correo).	Sistema	Validar datos del empleado

5. Diagrama de casos de uso de análisis

Este diagrama representa las funciones visibles para el Jefe de Logística dentro de TekMess. Siguiendo la estructura de la guía, la validación de datos se modela como un comportamiento obligatorio mediante una relación de inclusión (<<include>>)

.Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

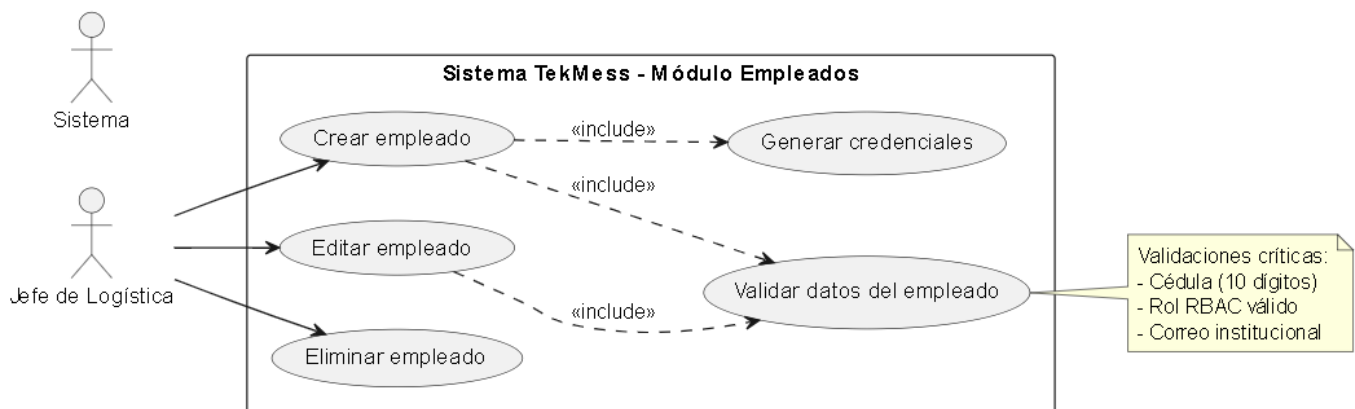
actor "Jefe de Logística" as Jefe
actor "Sistema" as Sistema

rectangle "Sistema TekMess - Módulo Empleados" {
    usecase "Crear empleado" as UC1
    usecase "Editar empleado" as UC2
    usecase "Eliminar empleado" as UC3
    usecase "Generar credenciales" as UC4
    usecase "Validar datos del empleado" as UC5
}

Jefe --> UC1
Jefe --> UC2
Jefe --> UC3

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>
UC1 ..> UC4 : <<include>>

note right of UC5
Validaciones críticas:
- Cédula (10 dígitos)
- Rol RBAC válido
- Correo institucional
end note
@enduml
```



6. Derivación de clases de análisis

A partir de los casos de uso del módulo RF-01: Gestionar empleado, identificamos las clases de análisis utilizando los estereotipos de frontera, control, entidad y repositorio

Esta separación asegura que la lógica de negocio esté aislada de la interfaz y de la base de datos PostgreSQL

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del Jefe de Logística y muestra resultados de la gestión	FormularioGestionEmpleado	Representa la pantalla con campos para Cédula, Nombres, Rol y Correo
Control	Coordina la lógica de registro, edición y la generación automática de credenciales	ControlEmpleado	Valida el formato de la cédula, el correo y gestiona el flujo de MFA/OTP
Entity / Entidad	Representa la información del personal operativo en el sistema	Empleado	Contiene los atributos críticos: cédula, nombres, rol (RBAC) y correo
Repositorio / Colección	Conserva y recupera los registros de empleados en la base de datos	RepositorioEmpleado	Permite guardar, buscar por cédula, actualizar y dar de baja al personal

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioGestionEmpleado" <<boundary>> {
    - txtCedula: String
    - txtNombres: String
    - txtCorreo: String
    - cbRol: RolRBAC
    + clickRegistrar()
    + clickActualizar()
    + clickDarBaja()
    + mostrarMensaje(msg: String)
    + listarPersonal(empleados: List)
}

class "ControlEmpleado" <<control>> {
    + crearEmpleado(datos): Resultado
    + editarEmpleado(cedula, datos): Resultado
    + darBajaEmpleado(cedula): Resultado
    + validarDatos(cedula, correo): boolean
    + generarCredenciales(): void
}

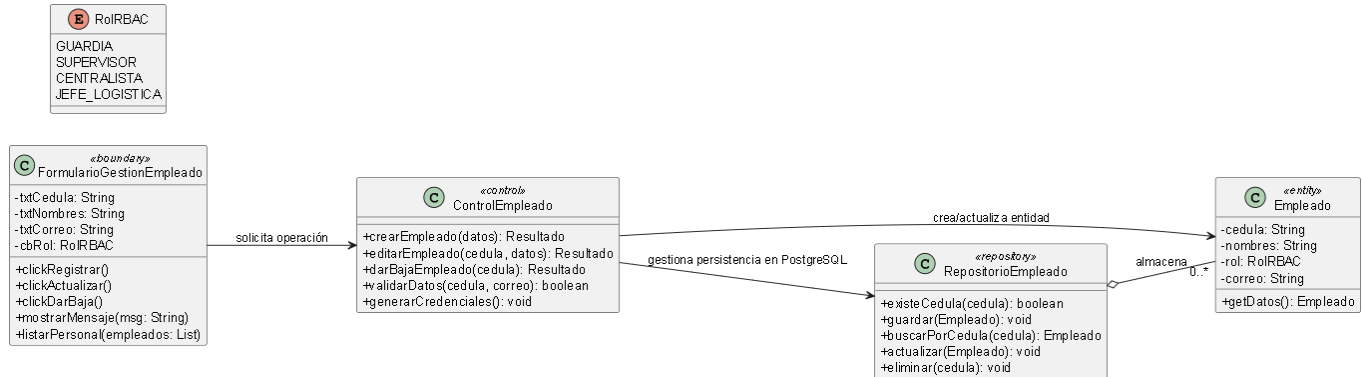
class "Empleado" <<entity>> {
    - cedula: String
    - nombres: String
    - rol: RolRBAC
    - correo: String
    + getDatos(): Empleado
}

enum "RolRBAC" {
    GUARDIA
    SUPERVISOR
    CENTRALISTA
    JEFE_LOGISTICA
}

class "RepositorioEmpleado" <<repository>> {
    + existeCedula(cedula): boolean
    + guardar(Empleado): void
}
```

```
+ buscarPorCedula(cedula): Empleado
+ actualizar(Empleado): void
+ eliminar(cedula): void
}
```

FormularioGestionEmpleado --> ControlEmpleado : solicita operación
 ControlEmpleado --> Empleado : crea/actualiza entidad
 ControlEmpleado --> RepositorioEmpleado : gestiona persistencia en PostgreSQL
 RepositorioEmpleado o-- "0..*" Empleado : almacena
 @enduml



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Crear empleado	Captura Cédula, Nombres, Rol y Correo del personal.	Valida formato de cédula y coordina el guardado	Crea la entidad Empleado y persiste en RepositorioEmpleado	Ingresar datos -> Validar -> Verificar cédula -> Crear entidad -> Guardar -> Confirmar.
Editar empleado	Carga datos actuales para modificación en el formulario.	Valida cambios y coordina la actualización de datos sensibles	Actualiza los atributos de la entidad Empleado en el repositorio.	Buscar empleado -> Cargar datos -> Validar cambios -> Actualizar -> Confirmar.
Dar de baja (Eliminar)	Solicita confirmación de desactivación de cuenta.	Verifica el estado actual y cambia el estado a "Inactivo"	Marca al Empleado como inactivo en el repositorio para revocar acceso.	Seleccionar empleado -> Confirmar baja -> Cambiar estado -> Notificar éxito.

8. Diagramas de secuencia de análisis

Estos diagramas muestran la colaboración dinámica de los objetos de TekMess a lo largo del tiempo para cumplir con las operaciones del RF-01

8.1 Secuencia: Crear empleado (Registro de personal operativo)

```
@startuml
title Secuencia de análisis - Crear Empleado (TekMess)

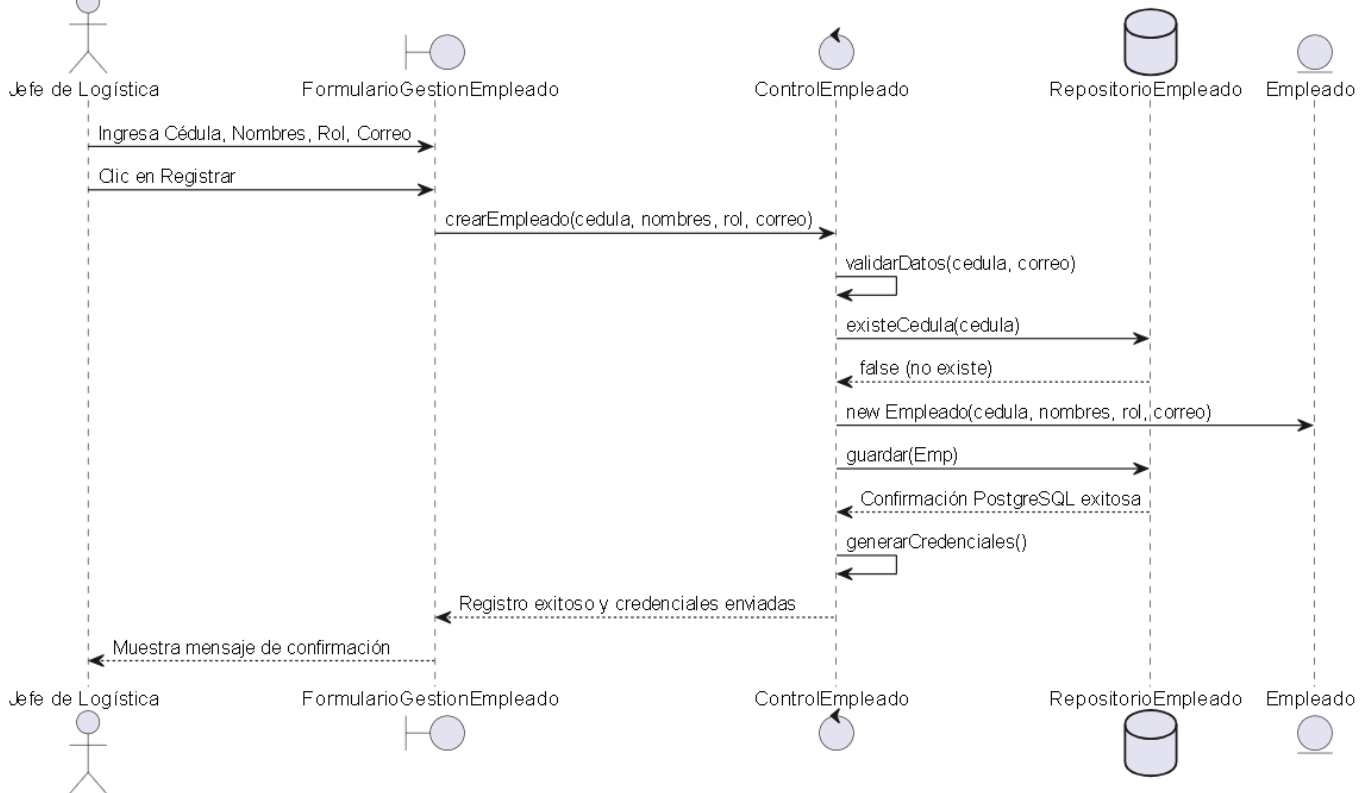
actor "Jefe de Logística" as Jefe
boundary "FormularioGestionEmpleado" as UI
control "ControlEmpleado" as Ctrl
database "RepositorioEmpleado" as Repo
entity "Empleado" as Emp

Jefe -> UI : Ingresar Cédula, Nombres, Rol, Correo
```

```

Jefe -> UI : Clic en Registrar
UI -> Ctrl : crearEmpleado(cedula, nombres, rol, correo)
Ctrl -> Ctrl : validarDatos(cedula, correo)
Ctrl -> Repo : existeCedula(cedula)
Repo --> Ctrl : false (no existe)
Ctrl -> Emp : new Empleado(cedula, nombres, rol, correo)
Ctrl -> Repo : guardar(Emp)
Repo --> Ctrl : Confirmación PostgreSQL exitosa
Ctrl -> Ctrl : generarCredenciales()
Ctrl --> UI : Registro exitoso y credenciales enviadas
UI --> Jefe : Muestra mensaje de confirmación
@enduml
    
```

Secuencia de análisis - Crear Empleado (TekMess)



8.2 Secuencia: Editar Empleado

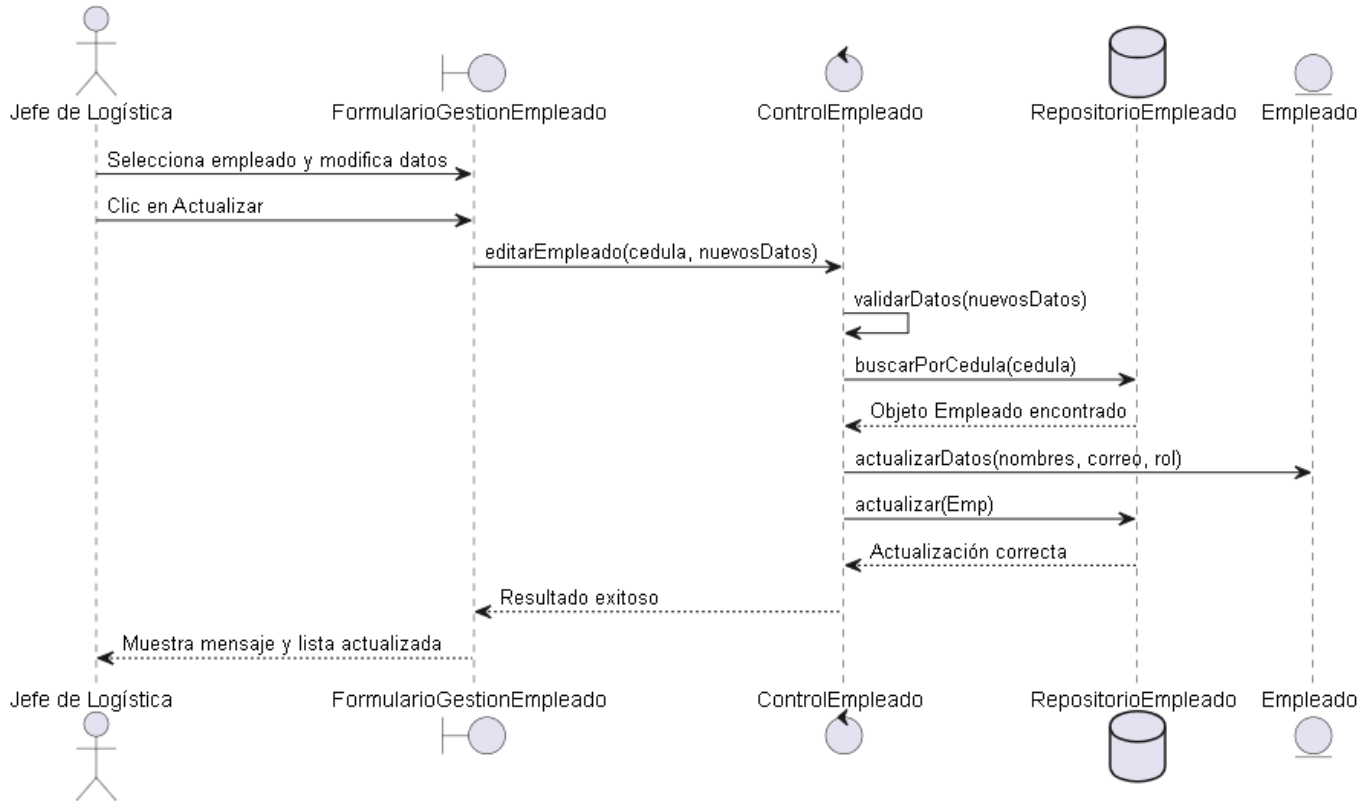
```

@startuml
title Secuencia de análisis - Editar Empleado (TekMess)

actor "Jefe de Logística" as Jefe
boundary "FormularioGestionEmpleado" as UI
control "ControlEmpleado" as Ctrl
database "RepositorioEmpleado" as Repo
entity "Empleado" as Emp

Jefe -> UI : Selecciona empleado y modifica datos
Jefe -> UI : Clic en Actualizar
UI -> Ctrl : editarEmpleado(cedula, nuevosDatos)
Ctrl -> Ctrl : validarDatos(nuevosDatos)
Ctrl -> Repo : buscarPorCedula(cedula)
Repo --> Ctrl : Objeto Empleado encontrado
Ctrl -> Emp : actualizarDatos(nombres, correo, rol)
Ctrl -> Repo : actualizar(Emp)
Repo --> Ctrl : Actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Jefe : Muestra mensaje y lista actualizada
@enduml
    
```

Secuencia de análisis - Editar Empleado (TekMess)



8.3 Secuencia: Eliminar empleado

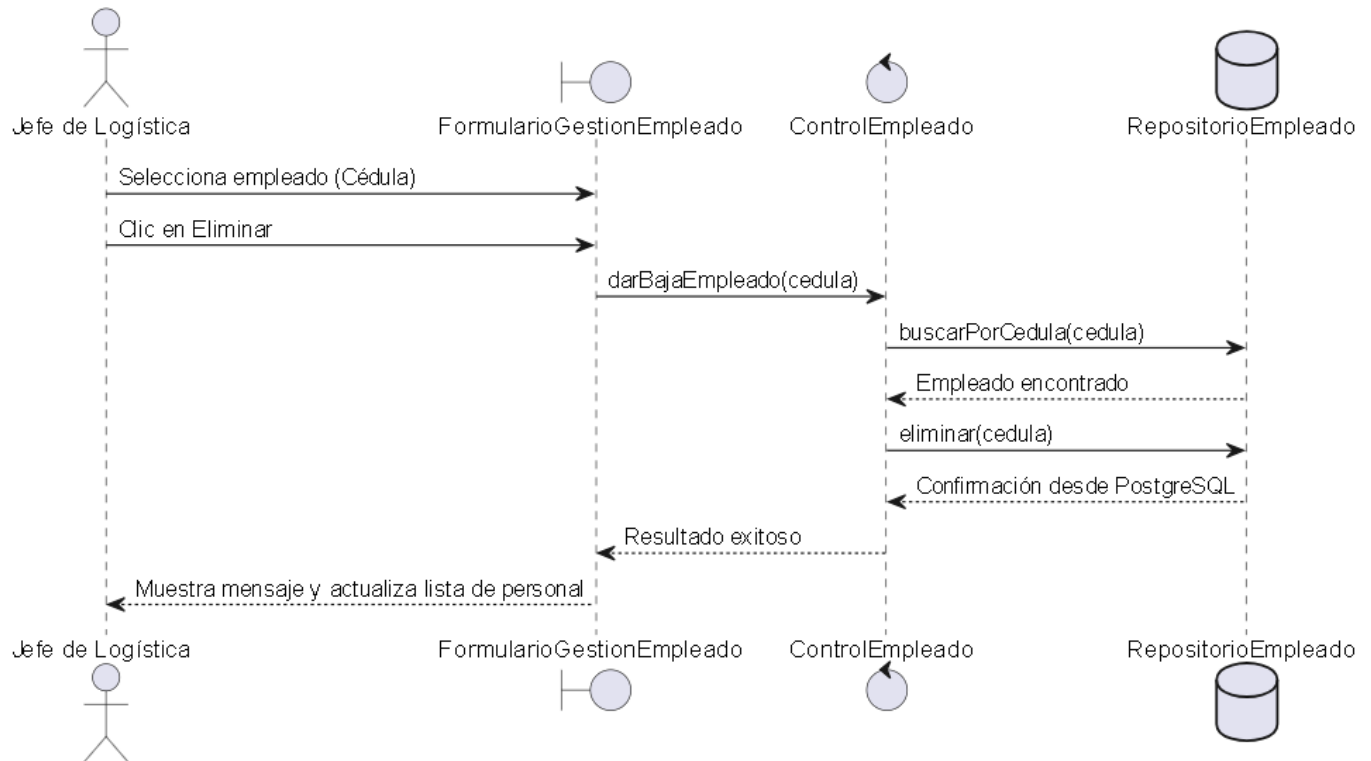
```

@startuml
title Secuencia de análisis - Eliminar Empleado (TekMess)

actor "Jefe de Logística" as Jefe
boundary "FormularioGestionEmpleado" as UI
control "ControlEmpleado" as Ctrl
database "RepositorioEmpleado" as Repo

Jefe -> UI : Selecciona empleado (Cédula)
Jefe -> UI : Clic en Eliminar
UI -> Ctrl : darBajaEmpleado(cedula)
Ctrl -> Repo : buscarPorCedula(cedula)
Repo --> Ctrl : Empleado encontrado
Ctrl -> Repo : eliminar(cedula)
Repo --> Ctrl : Confirmación desde PostgreSQL
Ctrl --> UI : Resultado exitoso
UI --> Jefe : Muestra mensaje y actualiza lista de personal
@enduml
    
```

Secuencia de análisis - Eliminar Empleado (TekMess)



8.4 Secuencia: Mostrar todos los empleados

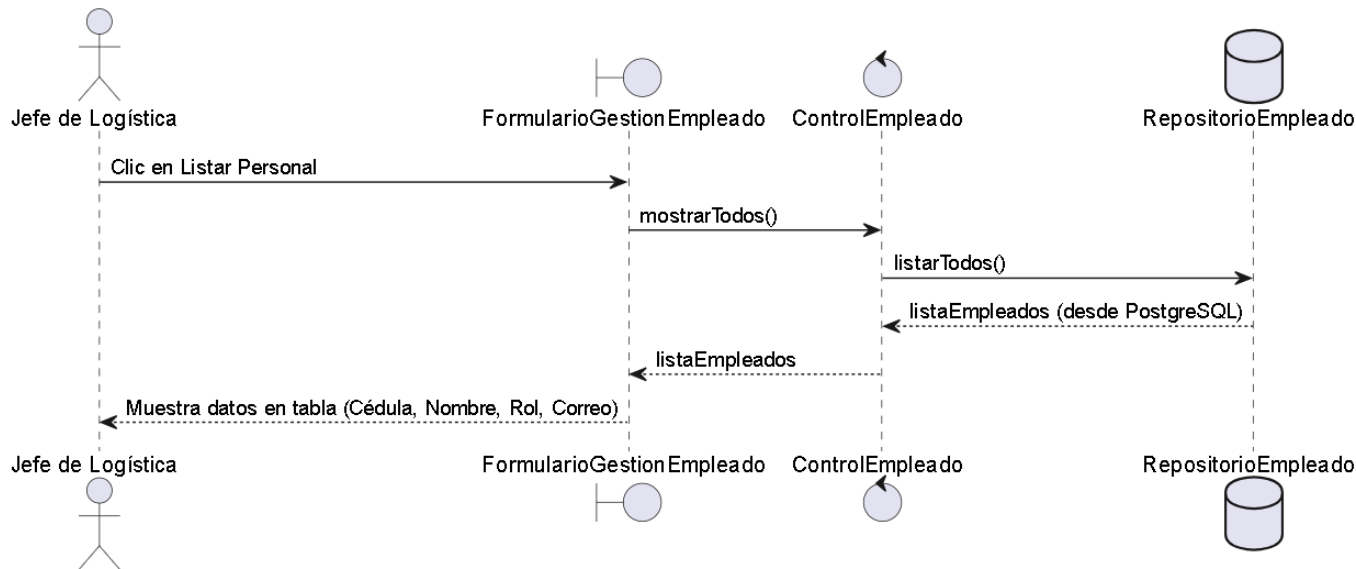
```

@startuml
title Secuencia de análisis - Mostrar empleados (TekMess)

actor "Jefe de Logística" as Jefe
boundary "FormularioGestionEmpleado" as UI
control "ControlEmpleado" as Ctrl
database "RepositorioEmpleado" as Repo

Jefe -> UI : Clic en Listar Personal
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEmpleados (desde PostgreSQL)
Ctrl --> UI : listaEmpleados
UI --> Jefe : Muestra datos en tabla (Cédula, Nombre, Rol, Correo)
@enduml
    
```


Secuencia de análisis - Mostrar empleados (TekMess)



9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Crear empleado	La Cédula ingresada no debe existir previamente en el sistema	Ingresar Cédula, Nombres, Rol y Correo -> Validar formato -> Verificar Cédula única -> Crear entidad -> Guardar en PostgreSQL -> Generar credenciales automáticas	Empleado registrado con éxito y notificación de acceso enviada al correo
Editar empleado	El empleado debe estar previamente registrado en la base de datos	Buscar empleado por Cédula -> Modificar datos permitidos (Nombres, Rol, Correo) -> Validar cambios -> Actualizar registro en el repositorio	Información del personal actualizada correctamente en el sistema
Eliminar empleado	El registro debe existir y el Jefe de Logística debe confirmar la acción de baja	Seleccionar empleado -> Confirmar eliminación -> Eliminar o inactivar registro en el repositorio -> Notificar éxito	El empleado es retirado de la lista activa y se revoca su acceso al sistema
Mostrar empleados	El usuario debe tener el rol de Jefe de Logística (RBAC)	Solicitar listado de personal -> El sistema recupera todos los registros del repositorio -> Presentar tabla de datos	Tabla actualizada visible con Cédula, Nombre y Rol de todo el personal operativo

10. Conclusión técnica

La implementación de la metodología de análisis para el módulo **Gestionar empleado de TekMess** demuestra que la relación entre los casos de uso, las clases de análisis y los diagramas de secuencia permite verificar que el sistema no se construye como un conjunto de piezas aisladas. En este proceso de modelado, los casos de uso definen qué acciones funcionales requiere el **Jefe de Logística**; las clases de análisis (Frontera, Control, Entidad y Repositorio) establecen qué objetos colaboran internamente; y los diagramas de secuencia ilustran la interacción dinámica necesaria para cumplir con cada operación de gestión.

Esta **trazabilidad técnica** asegura que el diseño sea plenamente coherente con los requisitos específicos de la **ERS**, garantizando que el control de acceso basado en roles (**RBAC**) para guardias, supervisores y centralistas, así como la persistencia de datos en **PostgreSQL**, se integren correctamente antes de iniciar la codificación en **Dart/Flutter**. Finalmente, este enfoque de diseño robusto permite que el sistema cumpla con normativas críticas como la **Ley Orgánica de Protección de Datos Personales**, minimizando el riesgo de errores humanos en la administración del personal operativo y preparando el camino para un desarrollo de software ordenado y escalable.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.